

MONDAY, AUGUST 31ST, 2026

OPERATIONS RESEARCH

The Patterns Never Written Down — Column Generation and the Cutting Stock Problem

There are 175,968 ways to slice this particular reel of paper, and the method that settles which ones to use will look at forty of them.

2,771 words · 15-minute read

A Reel, an Order Book, and Too Many Ways to Cut

A paper machine produces a continuous web of fixed width. At the end of the line a winder turns it into a master reel — 5,600 millimetres across, several tonnes, taller than the operator — and downstream of that sits a slitter with a bank of circular knives. Customers do not order 5,600 millimetre rolls. They order 423, and 767, and 901. The slitter's job is to convert one width into many, and the only question the mill gets to answer is where to put the knives.

A choice of knife positions is a *cutting pattern*: a list of ordered widths that together fit inside 5,600 millimetres. Four rolls of 591 and four of 809 come to exactly 5,600, and waste nothing. Two of 461, one of 513, one of 591, two of 637, one of 679, one of 767 and one of 853 come to 5,599, and waste a millimetre. Six of 901 come to 5,406, and throw away 194 millimetres of paper along the entire length of the reel, which for a reel some tens of kilometres long is a great deal of paper.

Consider a single day's order book: twelve distinct widths between 423 and 901 millimetres, 380 finished rolls in all, 247.97 kilometres of paper wanted in total. How many distinct patterns are available? The question has an exact answer, obtainable by a short dynamic program over the reel width, and the answer is 175,968. Of those, 77,245 are maximal, in the sense that no further ordered roll would fit in the space left over.

That is for twelve widths that are each a sixth or more of the reel. A mill slitting narrow rolls has a worse problem. Twenty ordered widths between 240 and 1,000 millimetres, off the same 5,600 millimetre reel, admit 27,559,358 patterns. Nothing about this is exotic; it is an ordinary Tuesday, and the catalogue of options is already past the point where anyone could store it, let alone read it.

The obvious way to model the problem makes this worse rather than better. Write down one variable for every (reel, ordered roll) pair — does roll j get cut from reel k ? — plus a variable saying whether reel k is used at all. This is essentially the formulation Leonid Kantorovich reached for in 1939, in the monograph that founded linear programming and that the West did not read until its English

translation appeared in 1960 [1]. It is compact, it is correct, and its linear relaxation is nearly useless, partly because every reel is interchangeable with every other, so the model spends its effort distinguishing between solutions that are the same solution wearing different labels.

Choosing the Variables Last

In 1961 Paul Gilmore and Ralph Gomory, both at IBM Research, published a reformulation that looks at first like a step backwards [2]. They had come to the problem the way industrial mathematics usually happens: visiting IBM customers to find out what those customers actually did all day, and finding at the paper mills a trim-loss problem nobody had solved properly [3].

Their model has one variable per *pattern*. Let a_{ip} be the number of rolls of width i that pattern p yields, let d_i be the number ordered, and let x_p be the number of master reels cut according to pattern p . Then

$$\min \sum_p x_p \quad \text{subject to} \quad \sum_p a_{ip} x_p \geq d_i \text{ for each width } i, \quad x_p \geq 0.$$

This is a linear program with twelve constraints and 175,968 variables. It has no symmetry, its relaxation is tight enough to be worth something, and it is completely impossible to write down.

The escape is to notice what the simplex method actually needs. At each step it does not require the constraint matrix; it requires *one column whose reduced cost is negative*, and a certificate that none exists when it terminates. Given dual prices y_i on the demand constraints, the reduced cost of pattern p is

$$\bar{c}_p = 1 - \sum_i y_i a_{ip}.$$

So the question “is there a column worth bringing in?” is the question “is there a feasible pattern a with $\sum_i y_i a_i > 1$?” — and finding the best such pattern is an optimisation problem in its own right:

$$z^*(y) = \max \left\{ \sum_i y_i a_i : \sum_i w_i a_i \leq W, a_i \in \mathbb{Z}_{\geq 0} \right\},$$

with w_i the ordered widths and W the reel width. That is an unbounded integer knapsack, and a dynamic program over the 5,600 possible remaining widths solves it in a few milliseconds. If $z^*(y) \leq 1$, no pattern anywhere in the catalogue prices out and the current solution is optimal for the full linear program, all 175,968 columns of it, without any of them having been examined. Gilmore and Gomory returned to the knapsack machinery in 1966 with a paper devoted to computing such functions fast [4].

The economics of the exchange is worth dwelling on. The master problem does not hand the pricing routine a list of patterns to score. It hands over a price per millimetre for each ordered width — the marginal value, today, given the current plan, of one more roll of that width — and asks a question a slitter foreman would recognise: at these prices, is there any way to set the knives that yields more than one reel’s worth of value from one reel? The linear program supplies the prices; the knapsack supplies the imagination.

Forty Columns

Start the mill instance from twelve trivial patterns, each cutting one width repeatedly. Solve, price, add the winning pattern, repeat.

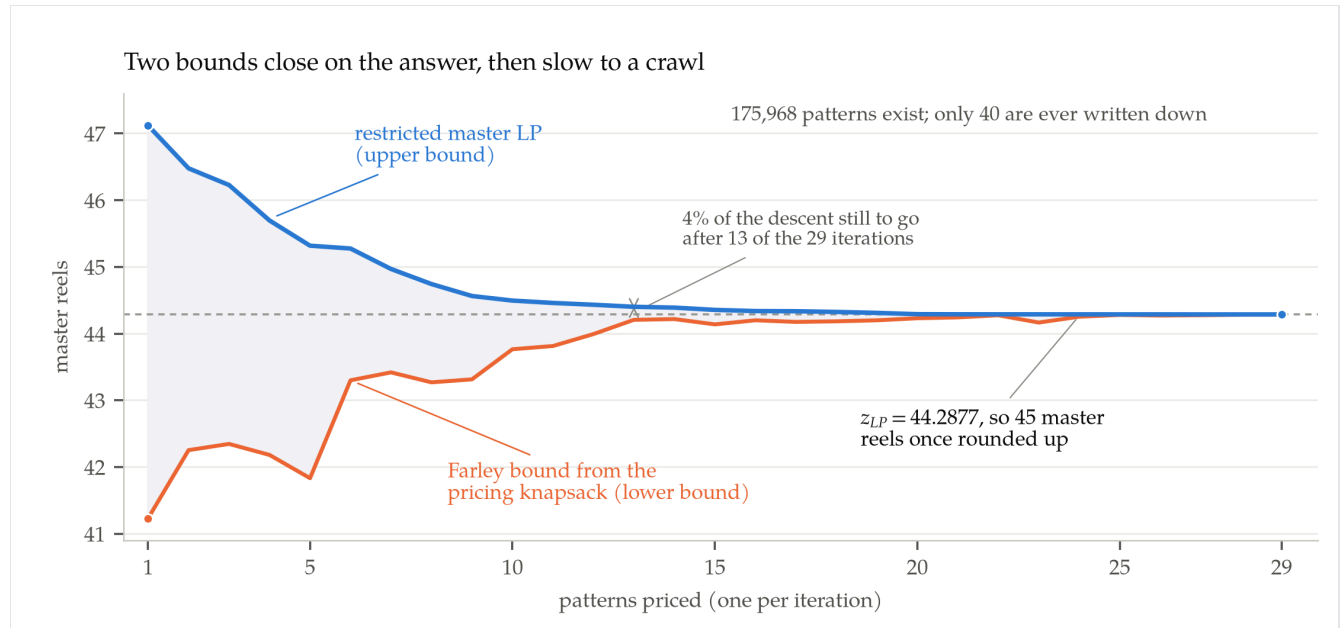


FIGURE Column generation on the mill instance. The restricted master's value falls monotonically from 47.12; Farley's bound rises, unevenly, from 41.23; they meet at 44.287709. The shaded region is the interval that still might contain the answer, and it takes twice as long to close the last four per cent as the first ninety-six.

Twenty-nine iterations later the pricing knapsack returns a best value of exactly one, and the process stops. Forty columns have been written down: the twelve it began with, plus twenty-eight invented along the way. That is one pattern in every 4,399 available. The linear program's optimum is 44.287709 reels — confirmed independently by solving the same linear program with all 175,968 columns present, which agrees to thirteen decimal places.

The convergence has two features worth naming. The first is the lower bound. A dual solution $\mathbf{y}$ from the restricted master is not feasible for the full dual, because some unpriced pattern may violate its constraint; but $\mathbf{y}/\mathbf{z}^*(\mathbf{y})$ is feasible, since $\mathbf{z}^*$ is by construction the worst violation. So $\mathbf{d} \cdot \mathbf{y}/\mathbf{z}^*(\mathbf{y})$, which is the restricted master's own value divided by the pricing optimum, is a valid lower bound on the true optimum at every iteration. This is Farley's bound, published as a two-page note in 1990 [5], and it is what turns column generation from a procedure that terminates into a procedure you can stop early with a guarantee.

The second is the shape. The first thirteen iterations capture 96 per cent of the descent; the remaining sixteen grind out the last four per cent. This is the phenomenon the literature calls tailing off, and the lower bound shows why it is not merely aesthetic: Farley's bound is not monotone. It rises to 44.21 at iteration 14, falls back to 44.14 at iteration 15, reaches 44.27 at iteration 22 and drops to 44.17 at iteration 23. The dual solution is jumping around between neighbouring optimal vertices of a degenerate

problem, and each jump sends the pricing routine hunting in a different direction. Damping that oscillation is the entire subject of stabilised column generation [6], and the jaggedness in the figure is what stabilisation is aimed at.

FORTY COLUMNS OUT OF 175,968 IS A RATIO THAT OUGHT TO BE SUSPICIOUS. WHAT LICENSES IT?

The licence is a fact about linear programs, not about paper. A linear program with m constraints has an optimal basic solution with at most m nonzero variables. This instance has twelve constraints, so however enormous the catalogue, the answer is supported on at most twelve patterns — and solving the fully enumerated linear program confirms it: exactly twelve of the 175,968 variables are nonzero at the optimum. Column generation is not sampling cleverly from a vast space. It is searching for twelve needles that are known in advance to be the only things there. The 40 columns are the 12 that matter plus the 28 the search had to look at to prove nothing better existed.

Pattern, widths in mm	Reels	Trim
4×591, 4×809	6.75	0
2×461, 1×513, 1×591, 2×637, 1×679, 1×767, 1×853	7.18	1
2×423, 2×461, 2×679, 1×767, 2×853	5.55	1
1×423, 1×767, 2×853, 3×901	5.10	1
6×513, 1×767, 1×853, 1×901	2.06	1

Five of the twelve patterns in the fractional optimum. Fractions of a reel are not cuttable, so the last step is to round. Solving the integer problem over the forty generated columns returns 45 reels, which is exactly **[44.2877]**; since no integer solution can beat the rounded-up relaxation, 45 is provably optimal over all 175,968 patterns, established without enumerating them. The mill cuts fifteen distinct patterns, discards 85 millimetres of edge trim in total across 45 reels, and overproduces four rolls it did not need. Restricting the same order book to single-width patterns — no mixing, one product per reel — costs 54 reels. Mixing saves one reel in six.

The Bound Is Nearly Worthless, Except When It Is Everything

There is a cruder lower bound available for nothing. Total ordered length divided by reel width says a solution cannot use fewer than 44.28 reels, because paper is conserved. How much does the linear program improve on that?

For this instance: by 0.007 of a reel. Seven-thousandths. All that machinery, and the relaxation lands almost exactly where division landed.

That is not a defect of the instance. Sweeping the same reel width against order books of different granularity makes a pattern visible, and it runs the opposite way to intuition.

Widths, mm	Pieces per reel	Patterns	LP minus material bound
201–461	17.0	58,509,987	0.0000
423–901	8.5	175,968	0.0070
1150–1900	3.7	405	1.8247
1901–2780	2.4	44	20.6121

The instances with astronomically many patterns are exactly the instances where the linear program tells you almost nothing beyond conservation of paper. The instance where the relaxation is worth twenty reels of information has forty-four patterns, and a patient person could write them all out on one sheet.

The reason is not mysterious once stated. When many pieces fit in a stock unit, there are so many near-perfect combinations that a fractional solution can pack the reel to within a rounding error of full, and the relaxation slides down onto the material bound. When only two or three pieces fit, the packing constraints bite: with eight widths all above 1,900 millimetres, no reel can hold three, so 250 ordered rolls require at least 125 reels no matter how the arithmetic of total length comes out. The relaxation discovers that and the material bound cannot. This is the regime where cutting stock stops being cutting stock and becomes bin packing.

The practical reading is that column generation’s product is usually the *patterns*, not the bound. What the mill wants is fifteen good knife settings, and the bound is a by-product that happens to certify them.

Rounding Up, and a Conjecture That Will Not Die

Rounding the relaxation up worked here. It works almost always. Across 600 randomly generated small instances solved to integer optimality by exhaustive enumeration, not one required more than the rounded-up relaxation, and the largest fractional shortfall observed was 0.971 — under one, every time.

Baum and Trotter gave this behaviour a name in 1981: the integer round-up property, or IRUP, holds for an instance when the integer optimum is less than one above the relaxation [7]. The natural conjecture is that cutting stock always has it. The natural conjecture is false. Odette Marcotte constructed a counterexample in 1986 [8], and the hunt since has been for how badly it can fail.

The answers are strange. Kartak, Ripatti, Scheithauer and Kurz settled the small cases exhaustively in 2015: every instance with at most nine distinct widths rounds up, proper counterexamples begin at ten distinct widths, and a gap exceeding one requires eleven [9]. Rietz and Dempe, constructing rather than searching, reached a gap of $13/11$ with eighteen distinct widths and $6/5$ with twenty-eight [10]. And

there the record stands. Nobody has ever exhibited a one-dimensional cutting stock instance whose integer optimum exceeds its relaxation by as much as 2. Scheithauer and Terno conjectured in 1995 that nobody ever will — the modified integer round-up property, MIRUP — and it remains open [11].

The theoretical side of the same question has moved further and stopped somewhere embarrassing. Karmarkar and Karp showed in 1982 that the relaxation can be turned into an integer solution using at most $O(\log^2 \text{OPT})$ extra bins [12]; Hoberg and Rothvoss improved the additive gap to $O(\log \text{OPT})$ in 2017 [13]. Those are magnificent results, and they are still unboundedly worse than the constant that everybody believes is true. Between an exhibited gap of $6/5$ and a proved bound that grows without limit lies one of the more durable embarrassments in combinatorial optimisation.

Everything Is a Column

Gilmore and Gomory’s paper is the one everybody cites, and Part II won the 1963 Lanchester Prize [3], [14]. It was not first. Three years earlier Ford and Fulkerson, working on maximal multi-commodity network flow, faced an arc-chain formulation with one variable per path and did precisely the same thing: they replaced the simplex method’s pricing step with a combinatorial routine that finds the shortest chain between two nodes, and never wrote the columns down [15]. Dantzig and Wolfe’s decomposition principle, which contains the general version of the manoeuvre, appeared in 1960 [16].

Even that ordering may be wrong. Uchoa and Sadykov argued in 2026 that a little-read 1951 book by Kantorovich and Zalgaller already contains a complete column generation algorithm — which would put the technique a decade before any of its usual parents, and in the hands of the person who is normally credited with the weaker formulation it replaced [17].

The idea generalises past recognition. Any integer program whose solutions decompose into repeated independent “objects” — patterns, routes, schedules, columns of a set-partitioning matrix — can be reformulated with one variable per object and priced instead of enumerated. An airline crew pairing problem covering four thousand flights admits billions of legal pairings; the master problem is a set partitioning model demanding each flight be covered once, and the pricing problem is a resource-constrained shortest path through a flight network subject to the duty-hour rules [18]. Vehicle routing works the same way, with routes for columns.

Making it exact for integer problems took another generation of work, because the obvious branching rule fails: forcing a single pattern variable to zero is a constraint the pricing subproblem cannot express, so the excluded column simply reappears. Barnhart and colleagues laid out the fixes in 1998 under the name branch-and-price, and the resulting scheme is now standard [19], [20]. An alternative route reaches the same bound with a formulation you *can* write down: Valério de Carvalho’s arc-flow model represents patterns as paths in a small graph, giving a compact model whose relaxation equals the Gilmore-Gomory one [21]. Delorme, Iori and Martello’s 2016 survey compares the descendants against each other on common instances [22].

Real mills, meanwhile, want things the model does not say. Kallrath and co-authors, reporting on production systems in the paper industry, describe overproduction limits, restrictions on how many distinct patterns a shift may set up, and multiple stock widths [23]; trim loss in a make-to-order mill has been reported at around five per cent before optimisation [24]. Each of these bends the pricing subproblem into something harder than a knapsack, and each of them is a live research problem.

But the move at the centre never changes, and it is a strange one to have discovered. Faced with a model too large to write down, do not look for a smaller model. Make the model larger — one variable per whole solution-object, tens of millions of them — until its structure is so uniform that a routine can manufacture the one variable you need on demand. The catalogue does not have to exist. It only has to be answerable.

REFERENCES

- [1] L. V. Kantorovich, "Mathematical Methods of Organizing and Planning Production," *Management Science*, vol. 6, no. 4, pp. 366-422, 1960; originally published in Russian, 1939.
- [2] P. C. Gilmore and R. E. Gomory, "A Linear Programming Approach to the Cutting-Stock Problem," *Operations Research*, vol. 9, no. 6, pp. 849-859, 1961.
- [3] INFORMS, "Gomory, Ralph E.," History of O.R. Excellence biographical profile, Institute for Operations Research and the Management Sciences.
- [4] P. C. Gilmore and R. E. Gomory, "The Theory and Computation of Knapsack Functions," *Operations Research*, vol. 14, no. 6, pp. 1045-1074, 1966.
- [5] A. A. Farley, "A Note on Bounding a Class of Linear Programming Problems, Including Cutting Stock Problems," *Operations Research*, vol. 38, no. 5, pp. 922-923, 1990.
- [6] O. du Merle, D. Villeneuve, J. Desrosiers and P. Hansen, "Stabilized column generation," *Discrete Mathematics*, vol. 194, pp. 229-237, 1999.
- [7] S. Baum and L. E. Trotter Jr., "Integer Rounding for Polymatroid and Branching Optimization Problems," *SIAM Journal on Algebraic and Discrete Methods*, vol. 2, pp. 416-425, 1981.
- [8] O. Marcotte, "An instance of the cutting stock problem for which the rounding property does not hold," *Operations Research Letters*, vol. 4, no. 5, pp. 239-243, 1986.
- [9] V. M. Kartak, A. V. Ripatti, G. Scheithauer and S. Kurz, "Minimal proper non-IRUP instances of the one-dimensional cutting stock problem," *Discrete Applied Mathematics*, vol. 187, pp. 120-129, 2015.
- [10] J. Rietz and S. Dempe, "Large gaps in one-dimensional cutting stock problems," *Discrete Applied Mathematics*, vol. 156, pp. 1929-1935, 2008.
- [11] G. Scheithauer and J. Terno, "The modified integer round-up property of the one-dimensional cutting stock problem," *European Journal of Operational Research*, vol. 84, no. 3, pp. 562-571, 1995.
- [12] N. Karmarkar and R. M. Karp, "An efficient approximation scheme for the one-dimensional bin-packing problem," in *Proc. 23rd Annual Symposium on Foundations of Computer Science*, pp. 312-320, 1982.
- [13] R. Hoberg and T. Rothvoss, "A Logarithmic Additive Integrality Gap for Bin Packing," in *Proc. 28th ACM-SIAM Symposium on Discrete Algorithms*, pp. 2616-2625, 2017.
- [14] P. C. Gilmore and R. E. Gomory, "A Linear Programming Approach to the Cutting Stock Problem—Part II," *Operations Research*, vol. 11, no. 6, pp. 863-888, 1963.
- [15] L. R. Ford and D. R. Fulkerson, "A Suggested Computation for Maximal Multi-Commodity Network Flows," *Management Science*, vol. 5, pp. 97-101, 1958.
- [16] G. B. Dantzig and P. Wolfe, "Decomposition Principle for Linear Programs," *Operations Research*, vol. 8, no. 1, pp. 101-111, 1960.

- [17] E. Uchoa and R. Sadykov, "Kantorovich and Zalgaller (1951): the o-th Column Generation Algorithm," *Mathematical Programming*, vol. 217, no. 1-2, pp. 439-456, 2026.
- [18] M. E. Lübbecke and J. Desrosiers, "Selected Topics in Column Generation," *Operations Research*, vol. 53, no. 6, pp. 1007-1023, 2005.
- [19] C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh and P. H. Vance, "Branch-and-Price: Column Generation for Solving Huge Integer Programs," *Operations Research*, vol. 46, no. 3, pp. 316-329, 1998.
- [20] F. Vanderbeck, "On Dantzig-Wolfe Decomposition in Integer Programming and ways to Perform Branching in a Branch-and-Price Algorithm," *Operations Research*, vol. 48, no. 1, pp. 111-128, 2000.
- [21] J. M. Valério de Carvalho, "Exact solution of bin-packing problems using column generation and branch-and-bound," *Annals of Operations Research*, vol. 86, pp. 629-659, 1999.
- [22] M. Delorme, M. Iori and S. Martello, "Bin packing and cutting stock problems: Mathematical models and exact algorithms," *European Journal of Operational Research*, vol. 255, no. 1, pp. 1-20, 2016.
- [23] J. Kallrath, S. Rebennack, J. Kallrath and R. Kusche, "Solving real-world cutting stock-problems in the paper industry," *European Journal of Operational Research*, vol. 238, no. 1, pp. 374-389, 2014.
- [24] R. Khan et al., "A Mathematical Model for Reduction of Trim Loss in Cutting Reels at a Make-to-Order Paper Mill," *Applied Sciences*, vol. 10, no. 15, art. 5274, 2020.

Daily Learning No. 015 · Operations Research · Monday, August 31st, 2026 · Sources are listed above; every load-bearing figure traces to a numbered reference.